GLO-2005 Modèles et langages des bases de données pour ingénieurs

Langage SQL : Les bases

Survol

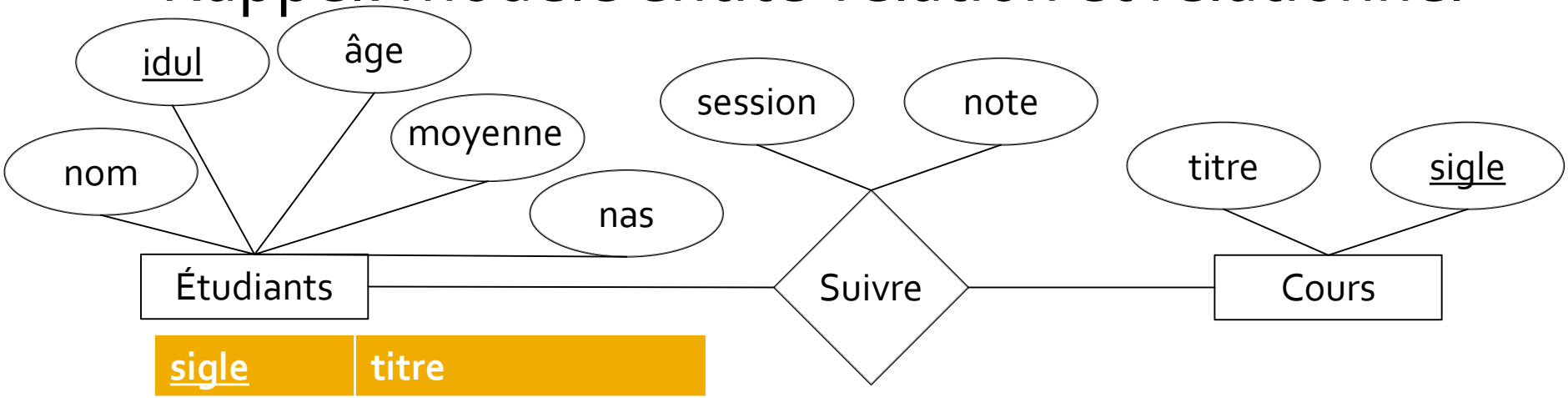
- Commandes de base
- Recherches dans les relations
  - Ramakrishnan & Gehrke : sections 5.1 à 5.6
  - Connolly & Begg : chapitre 6, sections 7.1, 7.3, 7.4

Introduction

- Langage de requêtes SQL
  - Développé par IBM dans les années 70
  - Implémentation approximative du modèle relationnel pour les SGBDR
  - Standard pour les bases de données aujourd'hui

Introduction

- Rappel: modèle entité-relation et relationnel



sigle	titre
GLO-2005	Bases de données
GLO-4035	BD avancées
GLO-4027	Données massives

idul	sigle	session	note
ALXYZ	GLO-2005	H17	87
BOUVW	GLO-2005	H17	75
CERST	GLO-2005	H16	88
ALXYZ	GLO-4035	A16	67
BOUVW	GLO-4027	A17	93

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	234567890
CERST	Cédric	22	3.1	345678901

Notation

```
CREATE [TEMPORARY] TABLE [IF NOT EXIST] nom
{ ( nom_attribut domaine [contrainte_attribut] [,…],
      [contraintes_relation [,…]] ) | LIKE ancien_nom }
[AS requête];
```

- Type de contenu
  - MAJUSCULES COURRIER COULEUR
    - Mot-clé
  - *minuscule italique*
    - Contenu variable à définir vous-même
- Type d'insertion
  - Contenu de la requête (incluant les parenthèses et le point-virgule) ;
    - Obligatoire
  - [Entre crochets]
    - Optionnel
  - [, …]
    - Répétez au besoin
  - { Entre accolades | séparé par un trait }
    - Liste de choix, obligatoire d'en choisir un

Notation

- Correspond à la notation de la documentation de MySQL
  - Les commandes sont simplifiées pour le cours

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
    (create_definition,...)
    [table_options]
    [partition_options]

CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
    [(create_definition,...)]
    [table_options]
    [partition_options]
    [IGNORE | REPLACE]
    [AS] query_expression

CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
    { LIKE old_tbl_name | (LIKE old_tbl_name) }

create_definition:
    col_name column_definition
  | {INDEX|KEY} [index_name] [index_type] (key_part,...)
      [index_option] ...
  | {FULLTEXT|SPATIAL} [INDEX|KEY] [index_name] (key_part,...)
      [index_option] ...
  | [CONSTRAINT [symbol]] PRIMARY KEY
      [index_type] (key_part,...)
      [index_option] ...
  | [CONSTRAINT [symbol]] UNIQUE [INDEX|KEY]
      [index_name] [index_type] (key_part,...)
      [index_option] ...
  | [CONSTRAINT [symbol]] FOREIGN KEY
      [index_name] (col_name,...)
      reference_definition
  | check_constraint_definition

column_definition:
    data_type [NOT NULL | NULL] [DEFAULT {literal | (expr)} ]
      [AUTO_INCREMENT] [UNIQUE [KEY]] [[PRIMARY] KEY]
      [COMMENT 'string']
      [COLLATE collation_name]
```

Création de relations

```
CREATE [TEMPORARY] TABLE [IF NOT EXIST] nom
{ ( nom_attribut domaine [contrainte_attribut] [,…],
      [contraintes_relation [,…]] ) | LIKE ancien_nom }
[AS requête];
```

- Si on supprime les portions optionnelles et on garde la base

```
CREATE TABLE nom ( nom_attribut domaine [,…] );
```

- Création de la relation Étudiants
  - `CREATE TABLE Etudiants ( idul char(5), nom char(40), age integer, moyenne double, nas integer );`
  - Le domaine de chaque attribut est spécifié, et le SGDBR impose cette contrainte sur chaque entité insérée dans la BD

Création de relations

- Domaines de valeurs communs
- `char(n)` : chaîne de caractères de longueur fixe *n*
- `varchar(n)` : chaîne de caractères de longueur variable jusqu'à un maximum de *n*
- `integer` ou `int` : entier
- `double` : valeur décimale flottante double-précision
- `boolean` : vrai ou faux (pas en MySQL; utiliser `int`)
- `date` : jour, mois, année, au format YYYY-MM-JJ
  - La fonction `CURDATE()` retourne la date actuelle
- `time` : heure, minutes, secondes, au format HH:MM:SS
  - La fonction `CURTIME()` retourne l'heure actuelle
- `timestamp` : date + time, au format YYYY-MM-JJ HH:MM:SS
  - La fonction `NOW()` retourne l'horodatage actuel
- `enum` : énumération de valeurs discrètes

Création de relations

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	234567890
CERST	Cédric	22	3.1	345678901

sigle	titre
GLO-2005	Bases de données
GLO-4035	BD avancées
GLO-4027	Données massives

idul	sigle	session	note
ALXYZ	GLO-2005	H17	87
BOUVW	GLO-2005	H17	75
CERST	GLO-2005	H16	88
ALXYZ	GLO-4035	A16	67
BOUVW	GLO-4027	A17	93

- ```
  CREATE TABLE Etudiants(
  idul char(5),
  nom char(40),
  age integer,
  moyenne double,
  nas integer );
  ```
- ```
  CREATE TABLE Cours(
  sigle char(8),
  titre varchar(100)
  );
  ```
- ```
  CREATE TABLE Suivre(
  idul char(5),
  sigle char(8),
  session char(3),
  note integer );
  ```
  ```

Insertion de tuples

```
INSERT INTO nom_relation { [( nom_attribut [,…] )]
VALUES (valeurs) [,…] | requête };
```

- On peut ajouter un tuple dans une relation
  - `INSERT INTO Etudiants(idul,nom,age,moyenne,nas) VALUES ('DIOPQ','Diane',26,3.3,456789012);`
- On n'a pas besoin de spécifier les attributs si
  - On entre des valeurs pour tous les attributs de la relation dans l'ordre de la définition de la relation
- On doit spécifier les attributs si
  - Tous les attributs de la relation ne sont pas inclus
    - Des attributs optionnels omis prennent une valeur par défaut
  - L'ordre des valeurs ne correspond pas à celui de la relation
    - On entre les attributs dans l'ordre qu'on veut

Recherche

- Le SQL est un langage non-procédural
  - On spécifie quelle information est demandée
  - L'optimisateur de requêtes du SGBDR se charge de trouver l'implémentation la plus efficace
- La commande de recherche est:

```
SELECT [DISTINCT] attribut [{INTO variable | AS nom}] [, ...]
FROM relation [, ...]
[ WHERE condition [AND ...] ]
[ GROUP BY attribut d'agrégation ]
[ HAVING critère d'inclusion ]
[ ORDER BY attribut [DESC]]
[ combinaison deuxième_requête ];
```

Recherche

```
SELECT [DISTINCT] attribut [{INTO variable | AS nom}] [, ...]
FROM relation [, ...]
[ WHERE condition [AND ...] ];
```

- Commande de base en trois parties
  - Liste de relations: toutes les relations impliquées dans la recherche
  - Liste d'attributs: Les attributs qu'on veut obtenir à partir des relations de la liste
    - DISTINCT: mot clé optionnel si on veut éliminer les duplicatas de la liste de cibles (par défaut ils sont conservés)
    - *: représente tous les attributs
    - AS: mot clé pour renommer un attribut
  - Liste de conditions: les critères à satisfaire pour retourner une valeur

Recherche

- Une condition a la forme:
  - *attribut opération* { *valeur* | *attribut* }
  - Opération :
    - op: <, >, =, <=, >=, <>, !=
    - op `ANY`, op `ALL`
    - `NOT` op
    - `IN` (= `ANY`), `NOT IN` (<> `ALL`)
    - `BETWEEN` minimum `AND` maximum (>= minimum `AND` <= maximum)
    - `LIKE` (pour chaînes de caractères)
    - `UNIQUE`, `EXISTS` (pour comparaison avec ensembles)
- La commande de recherche combine la sélection et projection en algèbre relationnelle
  - Jointure implicite lorsque plusieurs relations sont impliquées
  - Peut optionnellement renommer des attributs

Recherche

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	234567890
CERST	Cédric	22	3.1	345678901

- On veut voir tout le contenu de la relation
  - ```
    SELECT *
    FROM Etudiants;
    ```
- On veut trouver le nom de l'étudiant ayant l'IDUL CERST
  - ```
    SELECT nom
    FROM Etudiants
    WHERE idul = 'CERST';
    ```
  - $\pi_{nom}(\sigma_{idul=CERST}(E))$
- On veut toutes les informations des étudiants ayant plus de 3.5 de moyenne
  - ```
    SELECT *
    FROM Etudiants
    WHERE moyenne > 3.5;
    ```
  - $\sigma_{moyenne>3.5}(E)$
- On veut trouver le nom des étudiants plus vieux que 23 ans, et appeler cet attribut « vieillot »
  - ```
    SELECT nom AS vieillot
    FROM Etudiants
    WHERE age > 23;
    ```
  - $\rho_{vieillot/nom}\left(\pi_{nom}\left(\sigma_{age>23}(E)\right)\right)$

Recherche

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	234567890
CERST	Cédric	22	3.1	345678901

- On veut trouver le nom des étudiants ayant entre 20 et 30 ans
  - ```
    SELECT nom
    FROM Etudiants
    WHERE age BETWEEN 20 AND 30;
    ```
  - $\pi_{nom}\left(\sigma_{age \in [20,30]}(E)\right)$
- On veut trouver tous les étudiants dont le nom est Bob ou Cédric
  - ```
    SELECT *
    FROM Etudiants
    WHERE nom IN ('Bob', 'Cédric');
    ```
  - $\sigma_{nom \in \{Bob,Cédric\}}(E)$
- On veut trouver tous les étudiants dont le nom n'est pas Bob ou Cédric
  - ```
    SELECT *
    FROM Etudiants
    WHERE nom NOT IN ('Bob', 'Cédric');
    ```
  - $\sigma_{nom \notin \{Bob,Cédric\}}(E)$
    ```

Recherche

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	234567890
CERST	Cédric	22	3.1	345678901

- On veut trouver le nom et l'IDUL des étudiants ayant une moyenne de 3.5 ou plus
  - ```
    SELECT nom, idul
    FROM Etudiants
    WHERE moyenne >= 3.5;
    ```
  - $\pi_{nom,idul}\left(\sigma_{moyenne\geq 3.5}(E)\right)$
- On veut trouver le nom des étudiants ayant une moyenne de 3.5 ou plus et moins de 25 ans
  - ```
    SELECT nom
    FROM Etudiants
    WHERE moyenne >= 3.5 AND age < 25;
    ```
  - $\pi_{nom}\left(\sigma_{moyenne\geq 3.5,age<25}(E)\right)$
- On veut trouver le nom et l'IDUL des étudiants ayant une moyenne de 3.5 ou plus et moins de 25 ans
  - ```
    SELECT nom, idul
    FROM Etudiants
    WHERE moyenne >= 3.5 AND age < 25;
    ```
  - $\pi_{nom,idul}\left(\sigma_{moyenne\geq 3.5,age<25}(E)\right)$
    ```

Recherche

- Par défaut, la requête retourne les tuples résultats dans l'ordre qu'ils apparaissent dans la relation
  - Qui est l'ordre dans lequel ils ont été insérés dans la relation
- Si on veut ordonner les résultats, on ajoute une ligne à la requête

```
SELECT [DISTINCT] attribut [{INTO variable | AS nom}] [, …]
FROM relation [, …]
[ WHERE condition [AND …] ]
[ ORDER BY attribut [DESC]];
```

Recherche

- ```
  SELECT nom, age
  FROM Etudiants;
  ```

nom	age
Alice	24
Bob	27
Cédric	22

- ```
  SELECT nom, age
  FROM Etudiants
  ORDER BY age;
  ```

nom	age
Cédric	22
Alice	24
Bob	27

- ```
  SELECT nom, age
  FROM Etudiants
  ORDER BY age DESC;
  ```

nom	age
Bob	27
Alice	24
Cédric	22
  ```

Recherche

- On peut combiner plusieurs relations dans les recherches
- On veut le sigle de tous les cours dans lesquels Alice a eu plus de 80%
  - ```
    SELECT S.sigle
    FROM Etudiants E, Suivre S
    WHERE E.idul=S.idul AND E.nom='Alice' AND
          S.note>80;
    ```
  - $\pi_{sigle}\left(\sigma_{nom=Alice}(E) \bowtie_{idul} \sigma_{note>80}(S)\right)$

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	234567890
CERST	Cédric	22	3.1	345678901

idul	sigle	session	note
ALXYZ	GLO-2005	H17	87
BOUVW	GLO-2005	H17	75
CERST	GLO-2005	H16	88
ALXYZ	GLO-4035	A16	67
BOUVW	GLO-4027	A17	93
    ```

Recherche

- On veut tous les attributs des étudiants qui ont eu plus que 80% en GLO-2005
  - ```
    SELECT E.*
    FROM Etudiants E, Suivre S
    WHERE E.idul=S.idul AND S.note>80
          AND S.sigle='GLO-2005';
    ```
  - $\pi_{idul,nom,age,moyenne,nas}\left(E \bowtie_{idul} \sigma_{note>80 \wedge sigle=GLO-2005}(S)\right)$

idul	nom	age	moyenne	nas
ALXYZ	Alice	24	4.1	123456789
BOUVW	Bob	27	3.6	234567890
CERST	Cédric	22	3.1	345678901

idul	sigle	session	note
ALXYZ	GLO-2005	H17	87
BOUVW	GLO-2005	H17	75
CERST	GLO-2005	H16	88
ALXYZ	GLO-4035	A16	67
BOUVW	GLO-4027	A17	93
    ```

Recherche

- Dans cette requête, on a précisé les relations avec des lettres comme raccourcis
  - Nécessaire pour clarifier à quelle relation chaque attribut appartient
  - Permet une écriture plus compacte
- On aurait pu écrire les relations au complet
- Dans les premières requêtes, on aurait pu utiliser les raccourcis aussi
  - Pas obligatoire parce que c'est clair de quelle relation on parle
  - Mais c'est une bonne pratique

- ```
  SELECT S.sigle
  FROM Etudiants E,
       Suivre S
  WHERE E.idul=S.idul
   AND E.nom='Alice'
   AND S.note>80;
  ```
- ```
  SELECT Suivre.sigle
  FROM Etudiants,
       Suivre
  WHERE Etudiants.idul =
        Suivre.idul
   AND Etudiants.nom =
       'Alice'
   AND Suivre.note>80;
  ```
- ```
  SELECT E.nom
  FROM Etudiants E
  WHERE E.idul='CERST';
  ```
  ```

Recherche

- Recherche par chaîne de caractères
  - Opérateur `LIKE` pour trouver des chaînes qui suivent un patron
  - Opération _ pour représenter exactement un caractère arbitraire
  - Opération % pour représenter 0 ou plus caractères arbitraires

Recherche

- ```
  SELECT E.idul
  FROM Etudiants E
  WHERE E.nom = 'Alice';
  ```
  - Trouve les étudiants dont le nom est exactement « Alice »
- ```
  SELECT E.idul
  FROM Etudiants E
  WHERE E.nom LIKE 'Ali_e';
  ```
  - Trouve les étudiants dont le nom a la forme « Ali + (un caractère) + e »
  - Retourne « Alice », « Alise », « Aline », « Ali8e », etc.
- ```
  SELECT E.idul
  FROM Etudiants E
  WHERE E.nom LIKE 'A%e';
  ```
  - Trouve les étudiants dont le nom commence par A et se termine par E
  - Retourne « Alice », « Annie », « Alphonse », « A55e », « Ae », etc.
- ```
  SELECT E.idul
  FROM Etudiants E
  WHERE E.nom LIKE 'A___%e';
  ```
  - Trouve les étudiants dont le nom commence par A, se termine par E, et contient au moins quatre caractères
  - Retourne « Alice », « Annie », « Alphonse », « A55e », etc.

Recherche

- MySQL ignore les majuscules/minuscules et les accents dans les valeurs d'attributs
  - ```
    SELECT *
    FROM Etudiants E
    WHERE E.nom = 'Alice';
    ```
  - ```
    SELECT *
    FROM Etudiants E
    WHERE E.nom = 'ÀLICÉ';
    ```
    - Les deux requêtent retournent Alice
  - ```
    SELECT *
    FROM Etudiants E
    WHERE E.nom LIKE 'Céd%';
    ```
  - ```
    SELECT *
    FROM Etudiants E
    WHERE E.nom LIKE 'Ced%';
    ```
    - Les deux requêtes retournent Cédric

Recherche

- Attention! MySQL distingue les accents (mais pas les majuscules/minuscules) dans les noms de relations et d'attributs!
  - ```
    SELECT *
    FROM Etudiants E
    WHERE E.nom = 'Alice';
    ```
  - ```
    SELECT *
    FROM ETUDIANTS E
    WHERE E.NOM = 'Alice';
    ```
    - Les deux requêtent retournent Alice
  - ```
    SELECT *
    FROM Étudiants E
    WHERE E.nom = 'Alice';
    ```
    - Retourne une erreur : relation n'existe pas
  - ```
    SELECT *
    FROM Etudiants E
    WHERE E.nôm = 'Alice';
    ```
    - Retourne une erreur d'attribut inconnu

Attention! MySQL sur Linux distingue les majuscules/minuscules dans les noms de relations et d'attributs! Vous devez ajuster la configuration du serveur pour corriger ce comportement.

À suivre…

- La deuxième partie va plus en profondeur